



FACULTAD DE
ESTUDIOS PROFESIONALES
ZONA MEDIA



UNIVERSIDAD AUTÓNOMA DE SAN LUIS POTOSÍ

FACULTAD DE ESTUDIOS PROFESIONALES ZONA MEDIA

Práctica 7

**Contador Binario Ascendente de 4 Bits
mediante Pulsador
en Raspberry Pi Pico W**

Carrera:

Ingeniería Mecatrónica – Cuarto Semestre

Alumno:

Saúl Martínez Almazán

Docente:

Ing. Luis Jesús Padrón Padrón

Fecha:

25 de Febrero de 2026

Contents

1	Introducción	2
1.1	Objetivo	2
1.2	Trabajo Previo	2
1.3	Importancia del Contador Binario	2
2	Desarrollo	2
2.1	Configuración del Entorno de Desarrollo	2
2.2	Diseño del Circuito	3
2.3	Funcionamiento del Anti-Rebote por Software	3
2.4	Proceso de Compilación y Carga con PlatformIO	3
3	Resultados	4
3.1	Código Fuente: contador.cpp	4
3.2	Errores Comunes y Soluciones	6
4	Preguntas de Repaso	6
4.1	¿Qué pasa si <code>gpio_set_pulls(BUTTON, false, true)</code> se cambia por <code>gpio_set_pulls(BUTTON, true, false)</code> ?	6
4.2	¿Cuál es la función del cable rojo que sale del pin 40 al riel positivo de la protoboard?	7
4.3	Explica el bloque de código de las líneas 14 a 19 del archivo Contador.c	7
5	Conclusión	7
5.1	Reflexión	7
5.2	Mejoras Propuestas	8
5.3	Aplicación Futura	8
	Anexos	8

1 Introducción

En esta práctica se implementa un contador binario ascendente de 4 bits controlado mediante un pulsador, utilizando la placa Raspberry Pi Pico W programada con la extensión *PlatformIO* de Visual Studio Code. Cada pulsación incrementa en uno el contador, cuyo valor se representa en binario con 4 LEDs **rojos**. Al llegar al número 15 (1111 en binario), el conteo se reinicia automáticamente al cero.

1.1 Objetivo

Aprender a utilizar la resistencia interna de configuración *pull-down* de la Raspberry Pi Pico W, comprender el funcionamiento de un sistema anti-rebote (*debounce*) en pulsadores, e identificar correctamente los pines de propósito general y los pines de tierra en la placa.

1.2 Trabajo Previo

Como preparación a la práctica se consultó el capítulo 4 del documento *Raspberry Pi Pico-series C/C++ SDK*, específicamente la función `gpio_set_pulls` (sección 4.1.10.7.65). Esta función recibe tres parámetros: el número de pin GPIO, un valor booleano para activar o desactivar la resistencia interna *pull-up*, y un valor booleano para activar o desactivar la resistencia interna *pull-down*. Gracias a esta función es posible configurar el comportamiento del pin sin necesidad de agregar resistencias externas al circuito.

Adicionalmente, se configuró el entorno de desarrollo utilizando **PlatformIO** como extensión de Visual Studio Code, en sustitución del flujo CMake/SDK tradicional, lo que simplifica la gestión de dependencias y la compilación del proyecto.

1.3 Importancia del Contador Binario

Los contadores binarios son bloques funcionales fundamentales en sistemas digitales y embebidos. Su implementación en hardware mediante LEDs ofrece una representación visual directa del concepto de numeración binaria, y constituye la base de sistemas más complejos como temporizadores, controladores de secuencia y codificadores de posición.

2 Desarrollo

2.1 Configuración del Entorno de Desarrollo

En lugar del flujo convencional basado en CMake y el SDK oficial, se empleó **PlatformIO**, una extensión de Visual Studio Code que gestiona automáticamente las cadenas de herramientas (*toolchains*), dependencias y procesos de compilación para múltiples plataformas de microcontroladores, incluyendo la Raspberry Pi Pico W.

Ventajas de PlatformIO sobre CMake:

- **Gestión automática de dependencias:** No es necesario configurar manualmente `PICO_SDK_PATH` ni archivos `.cmake`.

- **Integración directa con VS Code:** Compilación, carga y monitor serial desde la misma interfaz gráfica.
- **Portabilidad:** El archivo `platformio.ini` reemplaza al `CMakeLists.txt` con una sintaxis más sencilla.

2.2 Diseño del Circuito

El circuito utiliza 4 LEDs **rojos** conectados a los pines GP6, GP7, GP8 y GP9 de la Raspberry Pi Pico W para representar los bits del contador (del más significativo al menos significativo, respectivamente). Un pulsador sin enclavamiento mecánico se conecta al pin GP15, el cual se configura con resistencia interna *pull-up*, de modo que el pin lee nivel alto en reposo y nivel bajo al presionar el botón.

Cada LED rojo cuenta con una resistencia limitadora de corriente de $200\ \Omega$. Con una caída de voltaje típica de 2.0 V para LEDs rojos y una alimentación de 3.3 V:

$$R = \frac{V_{CC} - V_{LED}}{I_{LED}} = \frac{3.3 - 2.0}{0.0065} \approx 200\ \Omega$$

Para el pulsador se utiliza una resistencia externa de $1\ \text{k}\Omega$ como protección adicional en serie con el pin de entrada, complementando la resistencia interna *pull-up* del microcontrolador.

2.3 Funcionamiento del Anti-Rebote por Software

Al presionar un pulsador mecánico se producen múltiples transiciones rápidas de voltaje (*rebotes*) antes de estabilizarse. Sin un mecanismo de filtrado, cada pulsación podría ser interpretada como varios eventos, incrementando el contador más veces de las esperadas.

La solución implementada consiste en:

1. Detectar el flanco de bajada del pin (transición de HIGH a LOW).
2. Esperar 50 ms (`delay(50)`) para que los rebotes se extingan.
3. Verificar nuevamente el estado del pin; si sigue en LOW, se confirma la pulsación válida y se incrementa el contador.
4. Esperar a que el botón sea liberado antes de aceptar una nueva pulsación, con un segundo *debounce* de 50 ms al soltar.

2.4 Proceso de Compilación y Carga con PlatformIO

1. **Creación del proyecto:** Desde la interfaz de PlatformIO se seleccionó la placa *Raspberry Pi Pico W* y el framework *Arduino*, generando automáticamente la estructura del proyecto.
2. **Escritura del código:** Se desarrolló el archivo fuente `main.cpp` con la lógica del contador (ver Sección 3).

3. **Compilación:** Se utilizó el botón *Build* de PlatformIO (o el atajo **Ctrl+Alt+B**). El entorno compila el proyecto y reporta errores en tiempo real.
4. **Carga del firmware:** Con la Pico W conectada en modo BOOTSEL, se utilizó el botón *Upload* de PlatformIO para transferir el binario directamente, sin necesidad de arrastrar manualmente el archivo `.uf2`.
5. **Verificación:** Tras la carga, el programa inicia de inmediato. Cada pulsación del botón incrementa el valor mostrado en los 4 LEDs en representación binaria, reiniciándose al superar el valor 15.

3 Resultados

3.1 Código Fuente: contador.cpp

A continuación se presenta el código fuente desarrollado para el contador binario ascendente de 4 bits:

```
1  #include <Arduino.h>
2
3  // --- Configuración de pines ---
4  // MSB -> GP6, luego GP7, GP8, LSB -> GP9
5  #define LED_MSB    6    // bit 3
6  #define LED_B2     7    // bit 2
7  #define LED_B1     8    // bit 1
8  #define LED_LSB    9    // bit 0 (LSB)
9
10 #define BTN_PIN    15    // Boton con pull-up interno (activo en
11                          // LOW)
12
13 #define DEBOUNCE_MS 50
14
15 const uint8_t leds[4] = {LED_MSB, LED_B2, LED_B1, LED_LSB};
16 uint8_t counter = 0;
17 bool last_btn_state = HIGH;
18
19 void actualizarLEDs(uint8_t valor) {
20     for (int i = 0; i < 4; i++) {
21         // leds[0] = MSB (bit 3), leds[3] = LSB (bit 0)
22         uint8_t bit = (valor >> (3 - i)) & 0x01;
23         digitalWrite(leds[i], bit);
24     }
25 }
26
27 void setup() {
28     for (int i = 0; i < 4; i++) {
29         pinMode(leds[i], OUTPUT);
30     }
31     pinMode(BTN_PIN, INPUT_PULLUP);
32 }
```

```
29     digitalWrite(leds[i], LOW);
30 }
31
32 // Pull-up interno: reposo HIGH, presionado LOW
33 pinMode(BTN_PIN, INPUT_PULLUP);
34 }
35
36 void loop() {
37     bool btn_state = digitalRead(BTN_PIN);
38
39     // Detectar flanco de bajada (boton presionado)
40     if (btn_state == LOW && last_btn_state == HIGH) {
41         delay(DEBOUNCE_MS);
42
43         // Confirmar que sigue presionado
44         if (digitalRead(BTN_PIN) == LOW) {
45             counter = (counter + 1) & 0x0F; // Limita el rango
46             // a 0-15
47             actualizarLEDs(counter);
48
49             // Esperar a que se suelte
50             while (digitalRead(BTN_PIN) == LOW) {
51                 delay(10);
52             }
53             delay(DEBOUNCE_MS); // Debounce al soltar
54         }
55     }
56
57     last_btn_state = btn_state;
58     delay(10);
59 }
```

3.2 Errores Comunes y Soluciones

Table 1: Errores encontrados y soluciones aplicadas durante el desarrollo

Error Encontrado	Solución Aplicada
El contador incrementa más de uno por pulsación.	Se implementó el mecanismo de <i>debounce</i> por software con doble lectura del pin y espera de 50 ms para filtrar los rebotes mecánicos del pulsador.
Los LEDs no muestran el valor correcto en binario.	Se verificó el orden de conexión de los LEDs rojos respecto al peso de los bits (MSB en GP6, LSB en GP9) y la lógica de desplazamiento de bits en <code>actualizarLEDs()</code> .
El botón no es detectado correctamente.	Se confirmó que el pin GP15 estuviera configurado como <code>INPUT_PULLUP</code> , que la resistencia de $1\text{ k}\Omega$ estuviera correctamente en serie, y que la lógica de detección fuera para flanco de bajada (LOW), ya que el botón es activo en nivel bajo.

4 Preguntas de Repaso

4.1 ¿Qué pasa si `gpio_set_pulls(BUTTON, false, true)` se cambia por `gpio_set_pulls(BUTTON, true, false)`?

La función `gpio_set_pulls(gpio, pull_up, pull_down)` recibe dos parámetros booleanos: el primero activa la resistencia interna *pull-up* y el segundo activa la resistencia interna *pull-down*.

Con la configuración original (`BUTTON, false, true`), la resistencia *pull-down* está activa: el pin lee cero lógico en reposo y uno lógico al presionar el botón (que conecta el pin a 3.3 V).

Al cambiarla a (`BUTTON, true, false`), se activa la resistencia *pull-up*: el pin lee uno lógico en reposo y cero lógico al presionar el botón (que lo conecta a GND). Esto **invierte la lógica de detección**: el código que antes detectaba una pulsación al leer un 1 ahora debería buscar un 0. De no adaptar el código, el contador incrementaría continuamente en reposo y se detendría al presionar el botón.

4.2 ¿Cuál es la función del cable rojo que sale del pin 40 al riel positivo de la protoboard?

El pin 40 de la Raspberry Pi Pico W corresponde al pin **VBUS**, que expone la tensión proveniente directamente del conector USB (nominalmente 5 V). El cable rojo traslada esta tensión al riel positivo de la protoboard, proporcionando una fuente de alimentación de 5 V disponible para alimentar componentes externos que así lo requieran (por ejemplo, sensores o módulos que no operan a 3.3 V). En este circuito también puede usarse como referencia de tensión para el pulsador si el diseño lo demanda.

4.3 Explica el bloque de código de las líneas 14 a 19 del archivo `Contador.c`

El bloque en cuestión corresponde a la función `actualizarLEDs()`:

```
1 void actualizarLEDs(uint8_t valor) {  
2     for (int i = 0; i < 4; i++) {  
3         uint8_t bit = (valor >> (3 - i)) & 0x01;  
4         digitalWrite(leds[i], bit);  
5     }  
6 }
```

Esta función recibe el valor actual del contador (un entero de 0 a 15) y lo representa en binario sobre los 4 LEDs. El bucle `for` itera cuatro veces, una por cada bit. En cada iteración:

- La expresión `(valor >> (3 - i))` desplaza el valor hacia la derecha el número de posiciones necesarias para colocar el bit de interés en la posición más baja.
- La operación `& 0x01` aplica una máscara que aísla únicamente ese bit (extrae un 0 o un 1).
- `digitalWrite(leds[i], bit)` escribe ese bit en el LED correspondiente: `leds[0]` recibe el bit más significativo (MSB) y `leds[3]` el menos significativo (LSB).

De este modo, si el contador vale 6 (0110 en binario), los LEDs mostrarán: apagado, encendido, encendido, apagado, de izquierda a derecha.

5 Conclusión

5.1 Reflexión

La implementación del contador binario ascendente de 4 bits consolidó conocimientos clave en el desarrollo de sistemas embebidos: uso de resistencias internas de configuración *pull-up/pull-down*, manejo de múltiples pines GPIO como salidas digitales, y diseño de un mecanismo robusto de anti-rebote por software. El uso de PlatformIO como entorno

de desarrollo simplificó considerablemente el proceso de compilación y carga, eliminando la necesidad de gestionar manualmente los archivos de configuración de CMake.

5.2 Mejoras Propuestas

- **Detección por interrupción:** Reemplazar el *polling* del botón por una interrupción por flanco (*IRQ*), lo que permite liberar al procesador y reducir el consumo energético.
- **Contador descendente:** Agregar un segundo pulsador para decrementar el contador, implementando un contador bidireccional.
- **Display de 7 segmentos:** Sustituir los LEDs individuales por un display de 7 segmentos para representar el valor en decimal, ampliando la utilidad educativa del circuito.
- **Anti-rebote por hardware:** Incorporar un circuito RC externo (resistencia-condensador) para complementar o reemplazar el *debounce* por software, aumentando la fiabilidad en entornos ruidosos.

5.3 Aplicación Futura

Los contadores binarios son componentes esenciales en una amplia variedad de sistemas: desde controladores de frecuencia y generadores de señales PWM, hasta sistemas de adquisición de datos y multiplexores de comunicación. El manejo de múltiples pines GPIO de forma sincronizada, junto con el control preciso de eventos de entrada, sienta las bases para el desarrollo de sistemas embebidos de mayor complejidad en aplicaciones de automatización e IoT.

Anexos

Anexo A – Código Fuente Completo

```
1 #include <Arduino.h>
2
3 // --- Configuración de pines ---
4 // MSB -> GP6, luego GP7, GP8, LSB -> GP9
5 #define LED_MSB 6 // bit 3
6 #define LED_B2 7 // bit 2
7 #define LED_B1 8 // bit 1
8 #define LED_LSB 9 // bit 0 (LSB)
9
10 #define BTN_PIN 15 // Boton con pull-up interno (activo en
11 // LOW)
12 #define DEBOUNCE_MS 50
13
```

```
14 const uint8_t leds[4] = {LED_MSB, LED_B2, LED_B1, LED_LSB};
15 uint8_t counter = 0;
16 bool last_btn_state = HIGH;
17
18 void actualizarLEDs(uint8_t valor) {
19     for (int i = 0; i < 4; i++) {
20         uint8_t bit = (valor >> (3 - i)) & 0x01;
21         digitalWrite(leds[i], bit);
22     }
23 }
24
25 void setup() {
26     for (int i = 0; i < 4; i++) {
27         pinMode(leds[i], OUTPUT);
28         digitalWrite(leds[i], LOW);
29     }
30     pinMode(BTN_PIN, INPUT_PULLUP);
31 }
32
33 void loop() {
34     bool btn_state = digitalRead(BTN_PIN);
35
36     if (btn_state == LOW && last_btn_state == HIGH) {
37         delay(DEBOUNCE_MS);
38
39         if (digitalRead(BTN_PIN) == LOW) {
40             counter = (counter + 1) & 0x0F;
41             actualizarLEDs(counter);
42
43             while (digitalRead(BTN_PIN) == LOW) {
44                 delay(10);
45             }
46             delay(DEBOUNCE_MS);
47         }
48     }
49
50     last_btn_state = btn_state;
51     delay(10);
52 }
```

Anexo B – Prototipo Armado

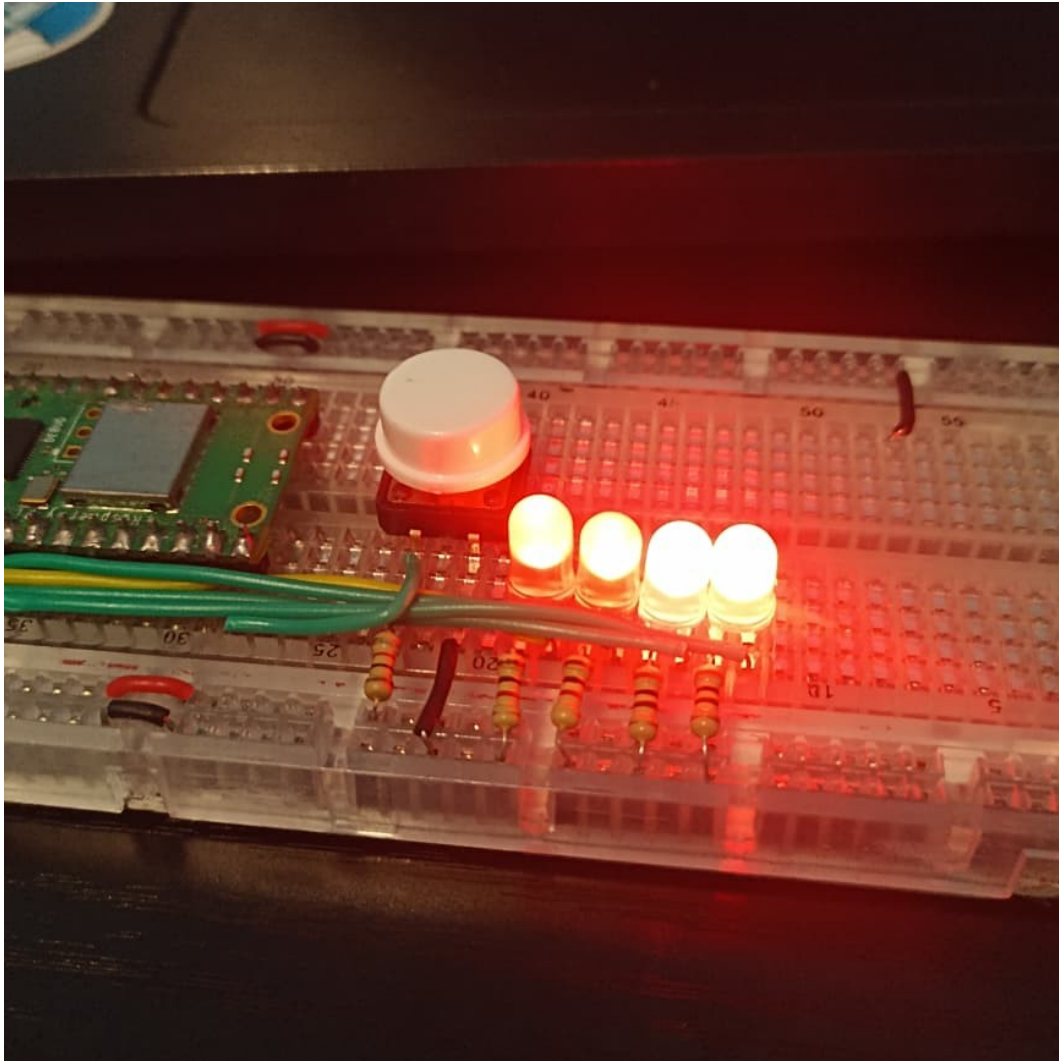


Figure 1: Fotografía del circuito armado en protoboard

References

- [1] Raspberry Pi Ltd., *Raspberry Pi Pico C/C++ SDK* (v2.0). <https://datasheets.raspberrypi.com/pico/raspberry-pi-pico-c-sdk.pdf>, 2024.
- [2] Raspberry Pi Ltd., *Raspberry Pi Pico W Datasheet*. <https://datasheets.raspberrypi.com/picow/pico-w-datasheet.pdf>, 2024.
- [3] Raspberry Pi Ltd., *RP2040 Datasheet*. <https://datasheets.raspberrypi.com/rp2040/rp2040-datasheet.pdf>, 2024.
- [4] PlatformIO Labs, *PlatformIO Documentation*. <https://docs.platformio.org>, 2024.